

milosvasic-ru

Miloš Vasić

Hero

AI engineer building verifiable AI-development systems.

I build the part of AI engineering that separates a dependable product from an impressive demo: the multi-provider LLM infrastructure that survives a provider going down, the autonomous agents and orchestration that keep the work on track, and the governance and QA layers that stop an AI system from quietly lying about what it can do. Turning a large language model into something you can actually ship is mostly a discipline problem, and that discipline is what I specialize in. My north star is a single rule — a feature is finished only when a real user can use it and there is captured evidence to prove it.

Summary

I work primarily in **Go**, with **Kotlin / Kotlin Multiplatform**, **TypeScript/React**, **Python**, **Swift**, and **Shell** as the job demands. What I care about most is how the work is *structured*: not a pile of one-off apps, but a fleet — large product applications sitting on top of dozens of small, decoupled, independently-tested modules, all inheriting a shared engineering **Constitution** as a Git submodule. That single architectural choice is what makes the whole body of work compound: fixes and improvements propagate across everything at once, new products assemble from parts that have already been proven, and every advertised capability is backed by an evidence-producing test rather than a claim. It is engineering built to be trusted at scale by one person. This page moves from that overview down to the individual projects; each links to its full product page.

How I work — governance and QA first

Before the products, the discipline that underpins them:

- **HelixConstitution** — I maintain a universal, inheritable engineering rulebook, distributed as a Git submodule across a 140+-repository fleet. It codifies anti-bluff evidence gates, false-positive immunity, data/host safety, and coverage rules; projects may tighten but never weaken it, and each governance gate is paired with a mutation test that proves the gate itself works. → see the HelixConstitution product page.
- **HelixQA** — I build anti-bluff QA orchestration that runs written test banks and fully-autonomous, LLM-and-vision-driven QA sessions across Android, Android TV, Web, and Desktop, scoring a PASS only when it has captured runtime evidence. → see the HelixQA product page.

My work across the Helix family

The Helix line spans the AI-development lifecycle. In priority order:

- **HelixTrack** — a free-world JIRA alternative; the flagship of the Helix-Track line.
- **HelixAgent** — an ensemble LLM service with multi-round model debate and verification-based provider selection.
- **HelixCode** — a distributed AI development platform that divides work across SSH-managed workers with automatic checkpoint/rollback.
- **HelixLLM** — a single binary with six modes serving OpenAI- and Anthropic-compatible APIs over HTTP/3, with local llama.cpp inference and a scored fallback chain.
- **HelixCluster** — a distributed operating system for AI compute, from datacenter GPUs to edge handhelds.
- **LLMProvider** — one interface over 43 providers with circuit breakers, retries, and health baked in.
- **LLMOrchestrator** — one control plane for every headless CLI coding agent.
- **LLMsVerifier** — verify, monitor, optimize: the single source of truth for LLM/provider/verification metadata.
- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — agent memory, governed agent skills, spec-driven development, AI application building, verified book translation, zero-trust terminals, federated Git, zero-brick OTA updates, and self-hosted cloud gaming.

My work across vasic-digital utils

Product-grade tools I built and maintain (each has a full product page):

- **Catalogizer** — multi-protocol (SMB/FTP/NFS/WebDAV/local), encrypted, self-hostable media collection management with a Go/Gin API and React UI.
- **Courses-Creator** — a markdown-to-video course pipeline with multi-LLM enrichment, TTS, and desktop/mobile/web players.
- **VisionEngine** — a decoupled Go toolkit fusing classic computer vision with multi-provider LLM vision for UI analysis and navigation graphs.
- **DocProcessor** — turns documentation into a verifiable feature map for QA automation (LLM or heuristic extraction).
- **Docs Chain** — a content-hashed, bidirectional, atomic document/DB sync engine.
- **Herald** — reliable multi-channel notifications with natural-language, three-tier intent resolution.
- **task_bridge** — a decoupled, bidirectional task/board sync engine (P1 scaffold; sync logic in progress).
- **Vasic Digital Reusable Module Suite** — the `digital.vasic.*` “standard library” of infrastructure, AI-primitive, and guardrail modules.

Infrastructure heritage (Server Factory)

Predating the AI line, my DevOps toolchain: **Mail Server Factory** (declarative JSON → fully-provisioned Dockerized mail servers, reporting 439 passing tests and a clean SonarQube gate), the **Server Factory Core Framework** it builds on, and VM-image tooling (**Qemu-Utills**, **Parallels-Utills**) plus supporting service factories.

In one line

I don't ship green checkmarks — I ship AI systems with the evidence that they actually work, and the governance that keeps them that way.

Contact

Open to senior AI/platform engineering roles worldwide.

- **Email:** milos85vasic@gmail.com · i@mvasic.ru
- **GitHub:** [milos85vasic](https://github.com/milos85vasic)

- **Telegram:** [@milos85vasic](https://www.telegram.me/milos85vasic)